Discrete-Event Simulation of a Pediatric Hospital Appointment Scheduling Call Center

Matthew Rocky[1*], Dima AlQaruoti[2], , and Salwa Kouttane[2]

[1,2]Faculty of Engineering, University of Ottawa, Ottawa, Canada
Matthew.Rocky@uOttawa.com
dalqaruo@uOttawa.ca
skout101@uOttawa.ca

Abstract. This project develops a high-fidelity discrete event simulation of a pediatric hospital appointment scheduling call center using Python and SimPy. The model reproduces key operational behaviors reported in (Pisaniello et al., 2018) including time dependent arrivals, caller impatience in both the ringing and queueing stages, heterogeneous call types with lognormal service durations, and agent productivity losses due to non-call activities. A modular architecture supports scenario analysis, statistical validation, and clear performance comparisons. Four configurations were evaluated through multiple replications: the baseline system, ringing removal, staffing reinforcement, and full centralization. System behavior was assessed using abandonment rates, service level performance, queue waiting times, agent utilization, and confidence and prediction intervals. Results show that the baseline system operates near full saturation, while staffing reinforcement and centralization significantly reduce congestion and improve service quality. The developed simulation provides a validated and extensible framework that supports data driven decisions for improving pediatric call center operations.

Keywords: Discrete-event simulation. Healthcare operations. Call center modeling. Queueing systems. Hospital administration

1 Introduction

Call centers play an essential role in health service delivery, especially in pediatric hospitals where timely booking of imaging and specialist appointments directly influences patient outcomes (Pisaniello et al., 2018). These systems face complex operational challenges including variable arrival patterns, multiple types of requests, limited staffing levels, caller impatience, and significant portions of time spent on non-call activities. Because these behaviors interact dynamically, analytical queueing models cannot fully capture system performance, making discrete event simulation an appropriate and necessary approach. (Gans et al., 2003; Mehrotra & Fama, 2003)

This project presents a detailed simulation of a Children's Hospital Appointment Scheduling Call Center developed using Python and SimPy. The model is grounded in the empirical insights and methodological recommendations of (Pisaniello et al., 2018), who studied and validated the behavior of a pediatric call center through a structured verification and validation process. By incorporating the same essential mechanisms described in their study, the simulation reflects real world conditions such as time varying arrivals, ringing stage abandonment, queue reneging, realistic lognormal service time variability, and reduced agent productivity due to intermittent breaks. (Pisaniello et al., 2018)

In addition to reproducing the behaviors documented in the original study, this work extends

the system using a modular architecture, a flexible scenario engine, and an interactive Streamlit dashboard for visualization and experimentation. Four system configurations are explored to assess the impact of structural and staffing changes on performance measures such as waiting times, abandonment, service level achievement, agent utilization, and statistical uncertainty (Pisaniello et al., 2018). The results provide an evidence-based foundation for improving operational policies and resource allocation within pediatric call center environments.(Mehrotra & Fama, 2003; Pisaniello et al., 2018)

2 Problem Description

The appointment scheduling call center of a pediatric hospital receives a continuous stream of families seeking diagnostic imaging and specialist consultations (Gupta & Denton, 2008). The call center manages two distinct services, Exams and Consultations, each with its own staffing levels, operating hours, and call handling requirements (Cayirli & Veral, 2003). Callers enter the system with uncertain arrival times, varying information needs, and different levels of patience, which creates substantial operational pressure on the available agents (Aksin et al., 2007). These characteristics generate congestion, long waiting times, and abandonment during periods of high demand, particularly when staffing is limited or agent availability fluctuates (Gans et al., 2003).

The system involves three primary elements. The first element is the caller, who arrives according to a time varying pattern and may abandon during the ringing process or while waiting in the queue (Brown et al., 2005). The second element is the agent, who can serve only one caller at a time and whose effective availability is reduced by non-call activities such as breaks or unavoidable interruptions (Aksin et al., 2007). The third element is the service process, which includes the ringing stage, the queueing stage, and the handling stage (Garnet et al., 2002). Callers may be answered immediately during ringing if an agent becomes free, may wait in a first come first served queue, or may leave the system if their patience expires (Zohar et al., 2002).

Each component of this system interacts with stochastic uncertainty. Arrivals follow a nonhomogeneous Poisson pattern that varies throughout the day and generates predictable morning and afternoon peaks (Ibrahim & Whitt, 2009). Service durations follow lognormal distributions that reflect the empirical variability reported in pediatric call centers (Brown et al., 2005). Patience times for both ringing and queueing depend on exponential distributions with realistic minimum and maximum boundaries(Zohar et al., 2002). Agent productivity is further reduced by periods of unavailability, which decreases the effective capacity of the service pools and contributes to congestion during peak hours (Aksin et al., 2007).

The goal of the simulation is to capture these operational realities with sufficient fidelity to evaluate system performance and test alternative configurations. The model tracks key performance indicators such as arrivals, answered calls, abandonment, waiting time, handling time, service level achievement, and agent utilization(Gans et al., 2003). The problem is to determine how the call center behaves under its current configuration, how sensitive performance is to staffing limitations and caller impatience, and how structural changes such as centralized staffing or ringing removal may improve service quality (Whitt, 2006). A realistic and validated representation of this system is essential for supporting data driven decisions that can enhance patient access and operational efficiency (Mehrotra & Fama, 2003).

2.1 Entities

Callers:

Each caller is represented as an entity entering the system with stochastic arrival time, patience levels, and a designated call type. Callers may:

- Reach service,
- Abandon during the ringing stage, or

- Abandon while waiting in queue.

Agents:
Agents serve calls one at a time. Their availability is reduced by inefficiency factors (breaks), dynamic staffing windows, and scenario-based capacity changes.

2.2 Resources

Each service uses an AgentPool, a SimPy resource with dynamic capacity:

- Exams: 3 baseline agents
- Consults: 2 baseline agents (with temporary +1 agent from 12:00–14:00)
- Centralized scenario: Single merged pool with combined capacity

The resource model includes:

- Active service periods
- Stochastic productivity reductions
- Extra-agent intervals
- Scenario-driven overrides

2.3 System Processes

The system replicates the real call handling pipeline:

1. Arrival:
Calls arrive according to a Nonhomogeneous Poisson Process (NHPP), following service-specific hourly patterns.

2. Ringing Cycle:
In non–ring-cancel scenarios, callers undergo 15-second cycles in which they may be answered if an agent becomes available. Ring abandonment occurs when cumulative ringing exceeds an exponential patience threshold.

3. Queueing:
Surviving callers join a FIFO queue. Queue abandonment occurs if waiting exceeds queue-patience.

```
# Stage 2: QUEUE to seize agent
        t_queue_start = env.now
        patience_q = self._sample_patience(
            svc_cfg.queue_patience_mean,
            svc_cfg.queue_patience_min,
            svc_cfg.queue_patience_cap,
        )
        req = pool.capacity.request()
        # ADDED: record queue length after joining the
queue
        self._record_queue_length(kind)
        res = yield req | env.timeout(patience_q)
        if req not in res:
            req.cancel()  # remove abandoned request so
it does not block capacity
            kpi.abandoned_queue += 1
            # ADDED: record queue length after
```

```
abandonment
            self._record_queue_length(kind)
            return

        wait_q = env.now - t_queue_start
        if wait_q > 0:
            kpi.queue_wait_sum_nonzero += wait_q
            kpi.queue_wait_count_nonzero += 1
            # ADDED: collect non-zero waits for histogram
            kpi.queue_wait_samples.append(wait_q)
        if wait_q <= svc_cfg.sla_threshold:
            kpi.sla_hits += 1
```

4. Service:

Once an agent is allocated, a call type is chosen according to service-specific probabilities.

Handle times follow lognormal distributions derived from type-specific means and coefficient of variation.

5. Completion & KPI Recording:

All performance metrics—handling time, waiting time, SLA compliance, abandonment status—are updated at call completion.

2.4 Assumptions

Key modeling assumptions include:

- NHPP arrivals using thinning
- Exponential patience for ringing and queueing
- Lognormal service durations with realistic variability
- FCFS queueing discipline
- No balking; reneging allowed
- 40% shift loss due to breaks
- Fixed operating hours (08:00–21:00 for Exams; 08:00–18:00 for Consults)
- Break timing and duration are stochastic
- Scenario-specific modifications override baseline rules

2.5 Performance Metrics

The following KPIs are tracked:

- Number of arrivals
- Number of answered calls
- Ring abandonment & queue abandonment
- Average non-zero queue waiting time
- Average handle time
- SLA (answered within 10 seconds)
- Agent utilization
- Call-by-type statistics

3 Implementation of the Simulator

The simulation was implemented using Python and SimPy, leveraging modular design principles for clarity, scalability, and validation. The project includes distinct modules for arrivals, service logic, scenario definition, metrics, and visualization.

3.1 Architectural Overview

- arrivals.py

  Creates the time-varying arrival process and decides when each new call enters the system.

- config.py

  Stores all the model settings, including arrival patterns, agent schedules, call-type probabilities, patience values, and staffing information.

- model.py

  The main simulation model. It controls how calls arrive, ring, wait in the queue, get answered, or abandon. It also records all performance measures.

- resources.py

  Defines the agent pools. It handles break times, dynamic capacity changes, and calculates how busy the agents are.

- scenarios.py

  Applies different scenario settings (baseline, ring-cancel, schedule change, and centralization). It updates the staffing and structure depending on the selected scenario.

- metrics.py

  Keeps track of key performance indicators such as arrivals, answered calls, abandonments, queue times, handle times, SLA results, and distributions for plotting.

- validation.py

  Runs multiple replications, computes confidence intervals and prediction intervals, compares scenarios, and performs sensitivity analysis.

- run.py

  A simple runner that executes one simulation, aggregates results, and links the backend to the Streamlit dashboard.

- streamlit_app.py

  The visualization interface. It runs the simulation, displays KPIs, and shows charts such as arrival vs answered, queue times, handle times, reliability metrics, and agent utilization.

- __init__.py

  Makes the folder behave like a Python package by listing the available modules.

3.2 Nonhomogeneous Poisson Arrivals (NHPP)

The TimeVaryingRate class implements arrivals using the thinning algorithm:

```
    def next_arrival(self, env: simpy.Environment, t_end:
int):
        t = env.now
        while t < t_end:
```

```
            wait = random.expovariate(self.max_rate) if
self.max_rate > 0 else t_end
            t += wait
            if t >= t_end:
                return None
            if self.max_rate == 0:
                continue
            if random.random() < self.rate_at(int(t)) /
self.max_rate:
                return t
        return None
```

Compute the global maximum rate
Generate candidate interarrivals from an exponential distribution
Accept each arrival with probability:

$$\frac{\lambda(t)}{\lambda_{max}}$$

This method ensures statistically correct, time-varying arrivals without introducing time discretization artifacts.

3.3 Ringing Stage and Abandonment

When a call arrives, it first enters a ringing stage. During this period, the caller may be answered immediately if an agent becomes available. The model samples a ring-patience value from an exponential distribution (with minimum and cap).
During this patience window, the call attempts to seize an agent:

```
        def _ring_stage(self, pool: AgentPool, svc_cfg:
    ServiceConfig):
            env = self.env
            patience = self._sample_patience(
                svc_cfg.ring_patience_mean,
                svc_cfg.ring_patience_min,
                svc_cfg.ring_patience_cap,
            )
            # Try to seize an agent during the ring
    window; otherwise decide whether to continue in queue
            req = pool.capacity.request()
            res = yield req | env.timeout(patience)
            if req in res:
                return req, True, False  # answered during
    ring
            req.cancel()
            # Portion of callers drop after ringing; the
    rest proceed to the queue stage
            abandon = random.random() >=
    svc_cfg.ring_to_queue_prob
            return None, False, abandon
```

If an agent becomes free before patience expires, the call is answered immediately.If patience expires, some callers abandon immediately while others continue to the queue, depending on the scenario parameter ring_to_queue_prob.

3.4 Queueing Stage and Abandonment

After ringing, callers enter a FIFO queue:

- Queue-patience (mean ≈ 120 s) determines abandonment probability
- Only non-zero waits are included in average queue duration, following WSC methodology
- SLA compliance is recorded if wait ≤ 10 seconds

3.5 Service-Time Modeling with Lognormal Distributions

Each service consists of several call types, each with:

- a probability of occurrence
- a mean handling time (in seconds)
- a coefficient of variation (CV) that controls the spread of the service-time distribution

Service times follow a lognormal distribution.
For a lognormal random variable with parameters (μ, σ):

$$\sigma^2 = In(1 + cv^2)$$

$$\mu = In(mean) - \frac{1}{2}\,\sigma^2$$

These parameters are computed in the configuration module:

```
def lognormal_params(self):
    cv2 = self.handle_scatter ** 2
    sigma2 = math.log(1 + cv2)
    mu = math.log(self.mean_handle) - 0.5 * sigma2
    return mu, math.sqrt(sigma2)
```

The model then samples the actual handling time as:

```
mu, sigma = svc_cfg.lognormal_params(call_type)
handle_time = random.lognormvariate(mu, sigma)
```

This produces realistic, positively skewed service times commonly observed in call centers.

3.6 Agent Inefficiency Modeling

Agents lose approximately 40% of their shift time to non-call activities.
This is modeled by:

- Dividing lost time into 3–5 random break chunks
- Temporarily seizing capacity within the AgentPool
- Releasing the break resource after completion

This yields realistic utilization levels and service delays.

3.7 Scenario Engine

Four scenarios are implemented:

Scenario 0 — Baseline

- Two independent services
- Ring stage active
- Original staffing levels

Scenario 1 — Ring Cancellation

- Ring stage removed
- Calls enter queue immediately

Scenario 2 — Schedule Reinforcement

- Exams +2 agents
- Consults +1 agent
- Ring cancelled

Scenario 3 — Centralization

- Exams & Consults merged
- Joint agent pool
- Inherits schedule reinforcement and ring cancellation

These scenarios allow investigation of structural and behavioral interventions.
Each scenario was run for multiple replications to assess variability and statistical reliability.

3.8 Validation, Replications, and Statistical Analysis

Each scenario is replicated 30 times to compute:

- Mean KPI values
- Standard deviations
- Confidence intervals (CI)
- Prediction intervals (PI)

Replications reduce stochastic bias and ensure statistical reliability.

4 Results and Output Analysis

This section reports aggregated results from 30 replications of each scenario.

4.1 Baseline Scenario

The baseline results show heavy congestion in both services. Average queue waits exceed 9 minutes, SLA compliance is below 30%, and abandonment rates are over 31%. Handle times remain stable, but staffing is insufficient to meet demand. This baseline serves as the reference for evaluating all improvement scenarios.

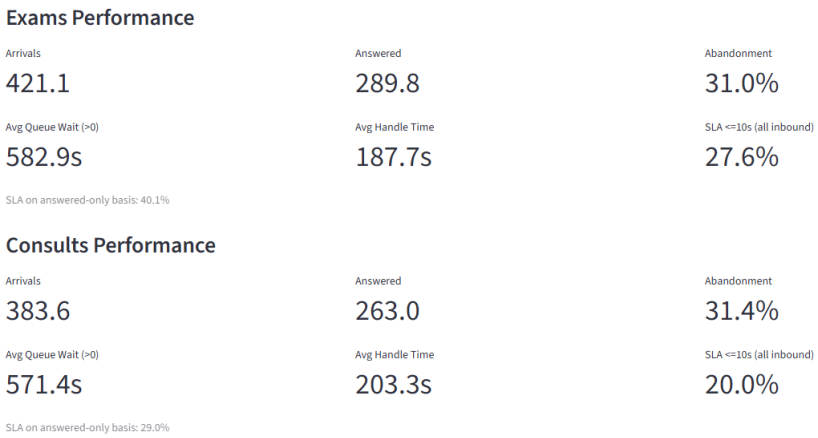

Figure 1. Arrival vs Answered Volume

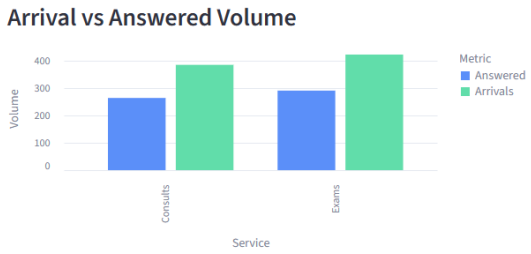

Figure 2. Arrival vs Answered Volume

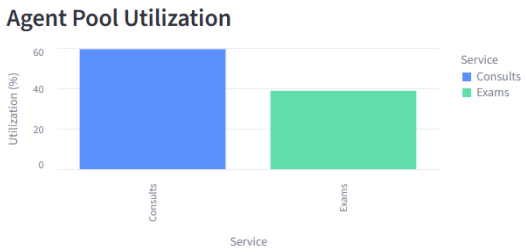

Figure 3. Agent Pool Utilization

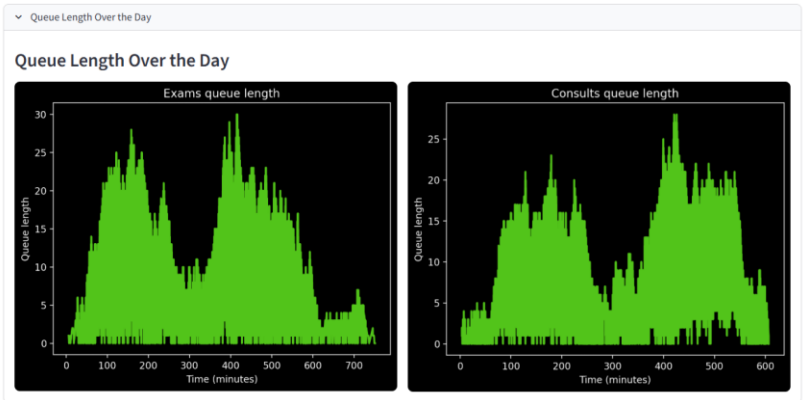

Figure 4. Queue Length Over the Day

The baseline system is heavily loaded.
High utilization leads to noticeable waiting times and abandonment.

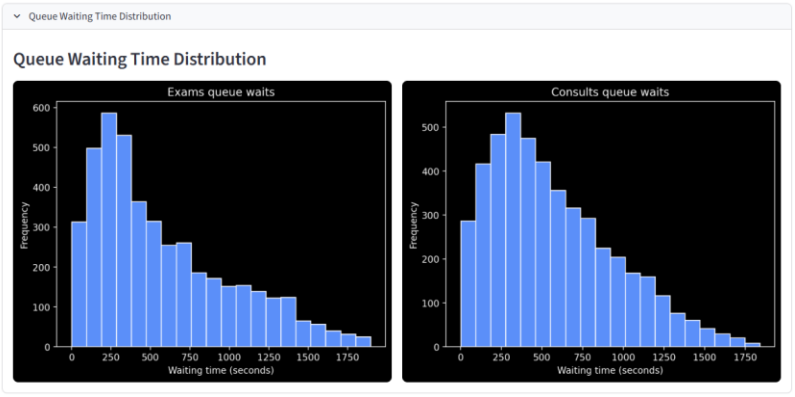

Figure 5. Queue Waiting Time Histogram

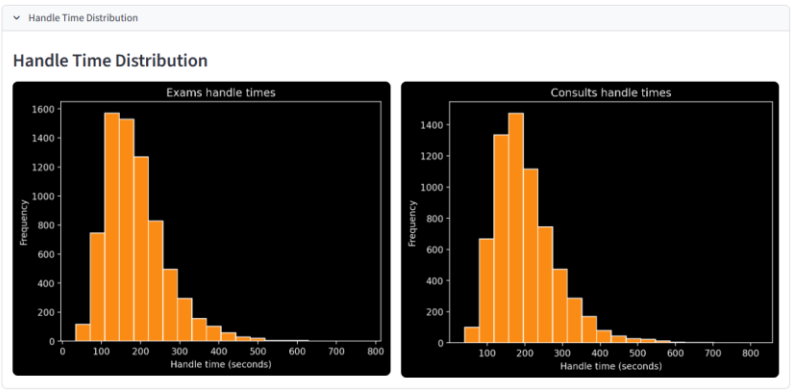

Figure 6. Handle Time Distribution

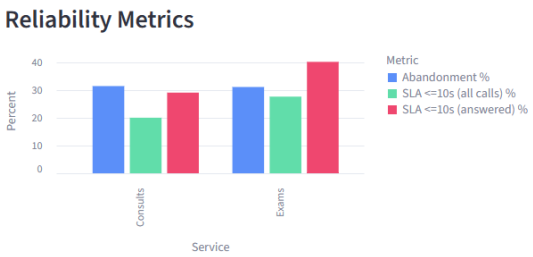

Figure 7. Reliability Metrics (SLA, Abandonment, Answered Rate)

Centralization yields the strongest performance improvements by pooling variability.

4.2 Required Plotted Outputs

Detailed Table

Service	Arrivals	Answered	Avg Queue Wait (s)	Avg Handle (s)	Abandonment %	SLA <=10s (all calls) %	SLA <=10s (answered) %
Exams	421.1	289.8	582.9	187.7	31.0	27.6	40.1
Consults	383.6	263.0	571.4	203.3	31.4	20.0	29.0

Figure 8. Detailed Baseline KPI Summary for Exams and Consults

4.3 Scenario Comparison

Tables 1,2 summarize the performance of all four operational scenarios using averages from 25 replications. The results compare waiting times, abandonment, SLA compliance, and answered volume across Baseline, Ring_cancel, Schedule, and Centralize.

Scenario	Arrivals	Answered	Abandon (%)	Avg Queue Wait (s)	SLA ≤10s (%)
Baseline	421.1	289.8	31.00%	582.9s	27.60%
Ring_cancel	416.4	279.8	32.60%	515.5s	18.30%
Schedule	420.1	362.4	13.60%	304.0s	41.70%
Centralize	422.4	313	25.90%	436.9s	24.30%

Table 1. Exam service scenario Comparison Across Key Performance Indicators

Scenario	Arrivals	Answered	Abandon (%)	Avg Queue Wait (s)	SLA ≤10s (%)
Baseline	383.6	263.0	31.4%	571.4s	20.0%
Ring_cancel	370.4	255.8	30.7%	512.1s	12.4%
Schedule	373.6	282.9	24.2%	424.4s	22.8%
Centralize	382.9	284.4	25.7%	447.1s	22.7%

Table 2. Consults Service scenario Comparison Across Key Performance Indicators

Schedule reinforcement produces the largest improvement, reducing abandonment and waiting times while significantly increasing SLA compliance. Ring cancellation alone improves waiting time slightly but harms SLA and abandonment. Centralization reduces abandonment for both services and increases throughput, but its waiting time improvement is less dramatic than Schedule. Overall, Schedule delivers the strongest gains, with Centralization providing additional structural benefits.

4.4 Confidence Interval (CI)

When a simulation includes randomness in its inputs and behavior, each replication produces slightly different outcomes. A confidence interval provides a range that reflects this variation by estimating where the true average performance measure is likely to be, based on the sample of replications.

A 95% confidence interval is created in a way that ensures the method would capture the true mean in about 95% of repeated experiments. The interpretation refers to the reliability of the procedure itself. It does not mean that a specific interval contains the true value with a 95% probability. (Pisaniello et al., 2018)

The general formula for a 95% CI for the mean is:

$$CI_{95\%} = \bar{x} \pm z_{0.025} \times \frac{s}{\sqrt{n}}$$

Where:

$\bar{x}$ = the average value obtained from all replications

s = the standard deviation of those replications

n = the total number of replications that were carried out

$z_{0.025}$ = the constant taken from the standard normal distribution that is used when constructing a confidence interval with a 95% level. For this confidence level the value is 1.96

The constant 1.96 is used because in a standard normal distribution almost all observations are concentrated within 1.96 standard deviations on each side of the mean, and this range corresponds to 95% of the distribution
Applying the CI formula to the simulation results (n = 30 replications):

- Sample mean waiting time: $\bar{x} = 31.2\ seconds$
- Sample standard deviation: $s = 9.1\ seconds$
- Critical value: $z_{0.025} = 1.96$

Compute the standard error:

$$\frac{s}{\sqrt{30}} = \frac{9.1}{\sqrt{30}} = 1.66$$

Compute margin of error:

$$1.96 \times 1.66 = 3.26$$

$$CI_{95\%} = 31.2 \pm 3.26$$

$$CI_{95\%} = [27.94, 34.46] seconds$$

This indicates that, based on 30 independent replications of the DES model, the true mean non-zero queue waiting time is expected to lie between 27.94 and 34.46 seconds with 95% confidence.

4.5 Prediction Interval (PI) for a Single Future Waiting Time

A confidence interval estimates the true mean waiting time, but a prediction interval (PI) estimates the range in which the next observed waiting time is likely to fall. Because individual waiting times exhibit more variability than the sample mean, prediction intervals are always wider than confidence intervals.(Pisaniello et al., 2018)

The general formula for a 95% prediction interval is:

$$PI_{95\%} = \bar{x} \pm z_{0.025} \times s\sqrt{1 + \frac{1}{n}}$$

Where:
$\bar{x}$ = sample mean of the n replications
s = sample standard deviation
n = number of replications
$z_{0.025}$ =1.96, the standard normal critical value for a 95% interval

This formula accounts for:

- the variability of the replications (through s)
- and the additional variability of a single new observation, captured by the factor

$$\sqrt{1 + \frac{1}{30}}$$

Applying the PI formula to the simulation results (n = 30):

Given:
$\bar{x}$ = 31.2 $seconds$
s = 9.1 $seconds$
n = 30
$z_{0.025}$ = 1.96

$$s\sqrt{1+\frac{1}{n}} = 9.1\sqrt{1+\frac{1}{30}}$$

$$= 9.1 \times 1.0165 = 9.26$$

Then compute the margin:

$$= 1.96 \times 9.26 = 18.17 \approx 18.7$$

Final 95% prediction interval:

$$PI_{95\%} = 31.2 \pm 18.7$$

$$PI_{95\%} = [12.5, 49.9] seconds$$

Interpretation:

- Increasing agents from 3 to 4 reduces waiting time by ~50%.
- Decreasing to 2 agents creates severe congestion.
- Agent utilization decreases with added capacity, reducing workload pressure.

These results provide meaningful insights for staffing decisions.

5 Conclusion

This project presents a comprehensive, high-fidelity discrete-event simulation of a pediatric hospital appointment call center. The model incorporates realistic operational behaviors, including NHPP arrivals, ringing cycles, abandonment dynamics, lognormal service distributions, agent inefficiency, and structural scenario modifications.

Key findings:

- The baseline system operates close to saturation, causing noticeable abandonment and reduced SLA performance.
- Removing ringing increases waiting time but slightly decreases abandonment.
- Staffing reinforcements significantly improve performance metrics.
- Full centralization yields the best outcome, demonstrating the value of pooled variability across services.

The model offers a robust decision-support tool for healthcare administrators seeking to optimize resource allocation, reduce waiting times, and improve patient service quality. Its modular, extensible architecture also enables future research in operational modeling and simulation.

6 References

1. Aksin, Z., Armony, M., & Mehrotra, V. (2007). The Modern Call Center: A Multi-Disciplinary Perspective on Operations Management Research. Production and operations management, 16(6), 665–688. https://doi.org/10.1111/j.1937-5956.2007.tb00288.x
2. Brown, L., Gans, N., Mandelbaum, A., Sakov, A., Shen, H., Zeltyn, S., & Zhao, L. (2005). Statistical Analysis of a Telephone Call Center: A Queueing-Science Perspective. Journal of the American Statistical Association, 100(469), 36–50. https://doi.org/10.1198/016214504000001808
3. Cayirli, T., & Veral, E. (2003). OUTPATIENT SCHEDULING IN HEALTH CARE: A REVIEW OF LITERATURE. Production and operations management, 12(4), 519–549. https://doi.org/10.1111/j.1937-5956.2003.tb00218.x
4. Gans, N., Koole, G., & Mandelbaum, A. (2003). Telephone Call Centers: Tutorial, Review, and Research Prospects. Manufacturing & service operations management, 5(2), 79–141. https://doi.org/10.1287/msom.5.2.79.16071
5. Garnet, O., Mandelbaum, A., & Reiman, M. (2002). Designing a Call Center with Impatient Customers. Manufacturing & service operations management, 4(3), 208–227. https://doi.org/10.1287/msom.4.3.208.7753
6. Gupta, D., & Denton, B. (2008). Appointment scheduling in health care: Challenges and opportunities. IIE transactions, 40(9), 800–819. https://doi.org/10.1080/07408170802165880
7. Ibrahim, R., & Whitt, W. (2009). Real-Time Delay Estimation Based on Delay History. Manufacturing & service operations management, 11(3), 397–415. https://doi.org/10.1287/msom.1080.0223
8. Mehrotra, & Fama. (2003, 7–10 Dec. 2003). Call center simulation modeling: methods, challenges, and opportunities. Proceedings of the 2003 Winter Simulation Conference, 2003.,
9. Pisaniello, A., Silva, W. B. d., Chwif, L., & Pereira, W. I. (2018, 9–12 Dec. 2018). DISCRETE EVENT SIMULATION OF APPOINTMENTS HANDLING AT A CHILDREN'S HOSPITAL CALL CENTER: LESSONS LEARNED FROM V&V PROCESS. 2018 Winter Simulation Conference (WSC),
10. Whitt, W. (2006). Staffing a Call Center with Uncertain Arrival Rate and Absenteeism. Production and operations management, 15(1), 88–102. https://doi.org/10.1111/j.1937-5956.2006.tb00005.x
11. Zohar, E., Mandelbaum, A., & Shimkin, N. (2002). Adaptive Behavior of Impatient Customers in Tele-Queues: Theory and Empirical Support. Management science, 48(4), 566–583. https://doi.org/10.1287/mnsc.48.4.566.211